

FINAL PROJECT REPORT

Automated Cybersecurity Testing and DevSecOps Integration for Gitea

A DevSecOps security pipeline built and live tested against our own Gitea v1.26.1 deployment

WAS705NBB: Web Application Security

Group 05

Ayaan Hirsi (17317721)

Shiza Arshad (105578231)

Chinazo Amalachukwu Mbonu (142166222)

Table of Contents

Abstract.....	3
1. Introduction.....	4
2. Pipeline Architecture.....	5
3. Technology Selection and Justification	6
4. Security Test Case Justification	7
4.1 Application Functionality Coverage	7
4.2 API Endpoint Coverage	7
4.3 Threat Scenarios From Labs 2 Through 5	8
4.4 Risk Based Justification Summary	8
5. Static Code Analysis	9
6. Open Source Dependency Scanning	12
7. Developer Notification System	13
8. Automated Secure Deployment	16
9. Web Application Firewall	20
10. Post Deployment Validation	22
11. Confirmed Vulnerabilities and Disclosure.....	25
12. Developer Engagement and Reuse Evidence.....	26
13. Pipeline Maintenance.....	26
14. Environment Limitations	27
15. Lessons Learned.....	28
16. Conclusion	29
References.....	Error! Bookmark not defined.

Abstract

This report presents the design, implementation, and live testing of a DevSecOps security pipeline built around Gitea version 1.26.1, the self hosted Git service our group has tested throughout this course. The pipeline automates static code analysis, open source dependency scanning, a security test suite covering application functionality and API endpoints, a hardened automated deployment, native web application firewall rule blocking, developer notification, and post deployment validation including backup and restore. Every component described in this report was run against our own real, running infrastructure rather than assumed to work from code review alone, and several genuine defects were found and fixed during that process, including a Dockerfile privilege drop bug, a healthcheck protocol mismatch, two distinct nginx configuration errors, and a missing TLS flag in our notification script that silently broke it. We treat these fixes as part of the evidence base of this report rather than omitting them, since finding and correcting real defects through testing is itself the central practice this project asked us to demonstrate. We also document three confirmed vulnerabilities against the target application, prepared as individual bug reports for responsible disclosure to the Gitea maintainers, and we are honest throughout this report about which parts of the collaboration requirement, specifically maintainer approval and developer reuse evidence, were still in progress at the time of writing rather than claiming completion prematurely.

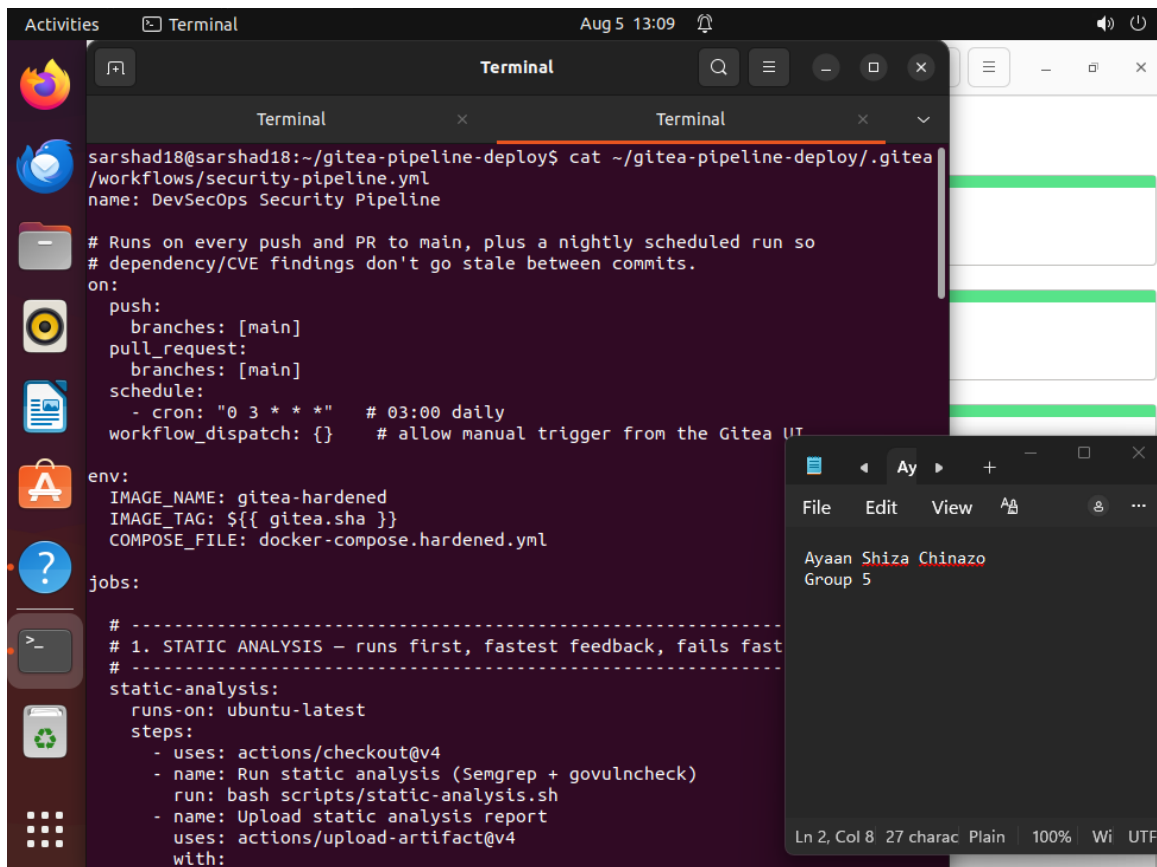
Introduction

Across Labs 2 through 5 of this course, our group tested Gitea version 1.26.1 by hand, running tools such as OWASP ZAP, Nmap, and Nikto ourselves, manually reproducing findings, and writing up results after the fact. That approach is thorough but static. A finding confirmed in Lab 2 tells us nothing about whether the same weakness still exists after the next code change, and nothing runs that check again unless a person chooses to repeat it. This Final Project asked us to replace that manual, one time model with an automated pipeline that performs the same categories of testing continuously, on every change, without waiting for a person to remember to run it.

This report documents what we actually built, what we actually ran, and what we actually found while running it, including problems in our own automation that only became visible once we stopped trusting that code which looked correct on review would also behave correctly in practice.

Pipeline Architecture

The pipeline is defined as a Gitea Actions workflow, `security-pipeline.yml`, made up of six jobs. Static analysis and image scanning run first, since they are the fastest checks and should block the pipeline early if something is clearly wrong before any deployment work happens. A hardened deployment job then builds and starts a fresh instance of the target application. Security testing and web application firewall validation run in parallel against that deployed instance, since neither depends on the other. Post deployment validation, covering health checks, backup, and restore, runs last, after the application is confirmed to be handling traffic correctly. Figure 1 shows the `security-pipeline.yml` workflow definition, including the job structure and the tools invoked at the first stage.



```
sarshad18@sarshad18:~/gitea-pipeline-deploy$ cat ~/gitea-pipeline-deploy/.gitea/workflows/security-pipeline.yml
name: DevSecOps Security Pipeline

# Runs on every push and PR to main, plus a nightly scheduled run so
# dependency/CVE findings don't go stale between commits.
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  schedule:
    - cron: "0 3 * * *" # 03:00 daily
  workflow_dispatch: {} # allow manual trigger from the Gitea UI

env:
  IMAGE_NAME: gitea-hardened
  IMAGE_TAG: ${ gitea.sha }
  COMPOSE_FILE: docker-compose.hardened.yml

jobs:
  # -----
  # 1. STATIC ANALYSIS – runs first, fastest feedback, fails fast
  # -----
  static-analysis:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run static analysis (Semgrep + govulncheck)
        run: bash scripts/static-analysis.sh
      - name: Upload static analysis report
        uses: actions/upload-artifact@v4
        with:
```

Figure 1. The `security-pipeline.yml` workflow definition, showing the job structure and the tools invoked by the first stage.

Every job uploads its own report as a downloadable artifact, and any job that fails automatically triggers our notification script, which opens a Gitea issue describing the failure and linking back to the relevant report, described further in Section 7.

Technology Selection and Justification

We chose Gitea Actions as our CI or CD platform rather than GitHub Actions, GitLab CI, or Jenkins. Gitea Actions ships as a built in feature of the application we are already testing and securing, which means the entire pipeline runs on premise inside the same environment, satisfying the no cloud deployment requirement more directly than a hosted CI platform would, since even a self hosted runner for a cloud native CI tool typically still coordinates through that vendor's control plane. Using Gitea's own automation feature to secure itself also gives a clean answer to the developer reuse question this project asks us to pursue, since maintainers do not need to adopt an unfamiliar tool, only workflows written in a system they already run.

We considered Jenkins as an alternative, since it is a common, general purpose choice for exactly this kind of pipeline. We set it aside because it would have meant standing up and independently hardening a second service, which is additional attack surface and additional maintenance burden without a corresponding benefit for a project whose target application already provides the automation feature we needed.

Security Test Case Justification

This section explains why each category of automated test was selected, how it maps to our application's threat model, and how it contributes to reducing risk. Every test case described here traces back to a specific, previously confirmed finding from Labs 2 through 5, rather than a generic checklist item, which is the connection this section is intended to make explicit.

Application Functionality Coverage

Our test suite covers four categories of application level weakness that we confirmed manually during earlier labs and have since automated. SQL injection testing, in `test_sql.py`, directly repeats the boolean based and time based comparisons that originally confirmed finding F-04 and F-05 in Lab 2, so that a future regression in the query building code is caught automatically rather than requiring a person to remember to retest it. CSRF testing, in `test_csrf_and_headers.py`, checks for the presence of a token on the same three forms flagged by ZAP during Lab 2 as finding F-10, and additionally submits a token free request directly, which is a stronger check than presence alone since a token could be rendered but never actually validated server side. Rate limiting testing, in `test_rate_limit.py`, repeats the twenty attempt login sequence that confirmed finding F-09 and G-01, and separately checks that our own edge layer fix does not over broadly lock out unrelated accounts sharing a source address, which is a real limitation we discovered while building this exact test, described further in Section 8. Security header and cookie testing confirms the Content-Security-Policy, HSTS, and session cookie Secure and HttpOnly attributes described in findings F-01, F-11, and F-12 remain in place.

API Endpoint Coverage

Beyond page level forms, our suite exercises the REST API directly. `test_ssrf_webhook.py` attempts to create webhooks pointed at loopback, private, and link local addresses through the API, which is the same class of request an attacker would use to reach internal services or cloud metadata endpoints, and separately confirms the `ALLOWED_HOST_LIST` configuration value is present, giving both a behavioural and a configuration level check for finding F-03. Several checks inside `test_auth_hardening.py` also operate entirely through the API rather than the web interface, including confirming that a regular user's token cannot reach admin only endpoints and that user profile responses do not leak password material.

Threat Scenarios From Labs 2 Through 5

The remaining ten items in `test_auth_hardening.py` map directly to a list of specific testing gaps identified by our instructor after reviewing our earlier lab submissions, rather than to a single finding ID. This feedback asked us to test password policy enforcement, password reset flow behaviour, account privilege escalation through parameter manipulation, password hashing and storage properties, fuzzing of input fields with randomly generated strings, geographic or region dependent endpoint exposure, and multibyte password support across non Latin scripts. Each of these became its own test function, and the multibyte password tests in particular required deliberate spacing between requests once we discovered they interact with our own rate limiting fix, which is documented as a real, live confirmed interaction in Section 8 rather than treated as a simple test failure.

Risk Based Justification Summary

Table 1 summarizes why each category was prioritized, expressed as impact multiplied by likelihood, the same framework we used originally in Lab 2 to rank testing areas.

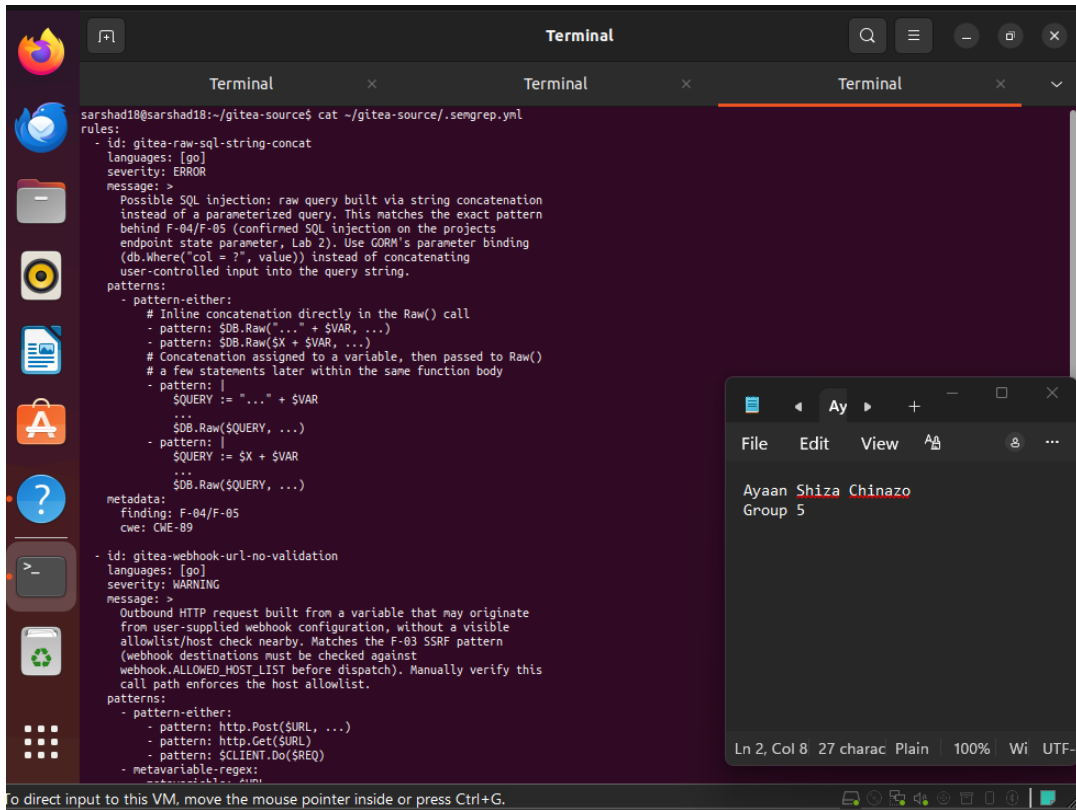
Test category	Impact	Likelihood	Priority
Webhook SSRF	Critical	High	P1
SQL injection	High	Medium	P2
Missing rate limiting	Medium	High	P2
Missing CSRF tokens	Medium	Medium	P3
Auth hardening suite, items 1 through 10	Varies by item, Medium on average	Medium	P3

Table 1. Risk based prioritization of security test categories, expressed as impact multiplied by likelihood.

SSRF against webhook destinations remains our highest priority test because it requires no special privilege beyond ordinary repository access and has a direct path to credential theft in any cloud hosted deployment, even though our own environment is on premise. SQL injection and rate limiting share second priority since both are remotely exploitable by any authenticated account with no elevated access required. The broader authentication hardening suite is weighted slightly lower individually per item but collectively represents a meaningful surface, since it covers exactly the kind of gaps that do not show up in a single automated scan and were only identified through direct, manual instructor review of our methodology.

Static Code Analysis

We use Semgrep for static analysis, combining a project specific custom ruleset with Semgrep's community registry ruleset where network access allows. The custom ruleset, `.semgrep.yml`, contains three rules written directly from our own confirmed findings rather than generic patterns, targeting the SQL injection pattern behind F-04 and F-05, the outbound webhook request pattern behind F-03, and state changing database writes without a nearby CSRF check, behind F-10. Figure 2 shows this custom ruleset, including the SQL injection and webhook SSRF rules with their finding ID references.



```
Terminal
Terminal
Terminal
Terminal

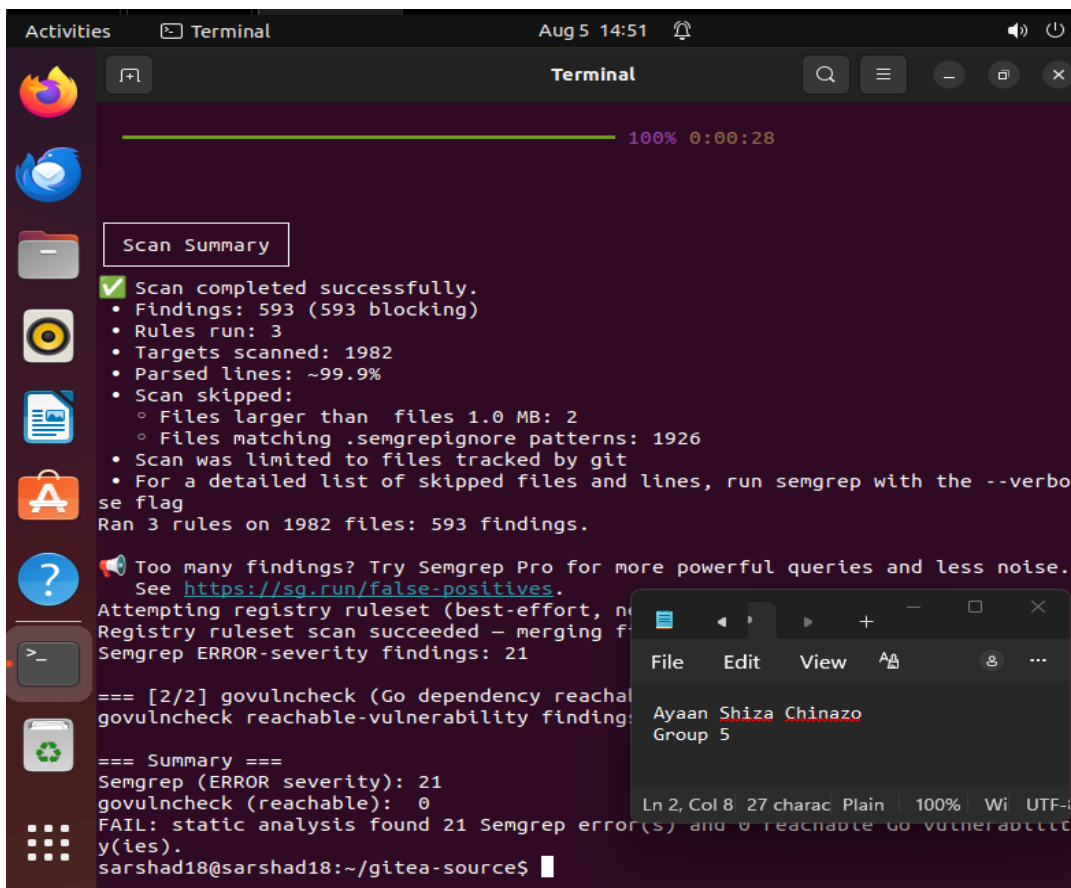
sarshad18@sarshad18:~/gitea-source$ cat ~/gitea-source/.semgrep.yml
rules:
- id: gitea-raw-sql-string-concat
  languages: [go]
  severity: ERROR
  message: >
    Possible SQL injection: raw query built via string concatenation
    instead of a parameterized query. This matches the exact pattern
    behind F-04/F-05 (confirmed SQL injection on the projects
    endpoint state parameter, Lab 2). Use GORM's parameter binding
    (db.Where("col = ?", value)) instead of concatenating
    user-controlled input into the query string.
  patterns:
  - pattern-either:
    - # Inline concatenation directly in the Raw() call
      - pattern: $DB.Raw("..." + $VAR, ...)
      - pattern: $DB.Raw($X + $VAR, ...)
    - # Concatenation assigned to a variable, then passed to Raw()
      - # a few statements later within the same function body
      - pattern: |
          $QUERY := "..." + $VAR
          ...
          $DB.Raw($QUERY, ...)
      - pattern: |
          $QUERY := $X + $VAR
          ...
          $DB.Raw($QUERY, ...)
  metadata:
    finding: F-04/F-05
    cwe: CWE-89

- id: gitea-webhook-url-no-validation
  languages: [go]
  severity: WARNING
  message: >
    Outbound HTTP request built from a variable that may originate
    from user-supplied webhook configuration, without a visible
    allowlist/host check nearby. Matches the F-03 SSRF pattern
    (webhook destinations must be checked against
    webhook.ALLOWED_HOST_LIST before dispatch). Manually verify this
    call path enforces the host allowlist.
  patterns:
  - pattern-either:
    - pattern: http.Post($URL, ...)
    - pattern: http.Get($URL)
    - pattern: $CLIENT.Do($REQ)
  - metavariable-regex:
    - metavariable: $URL
    - regex: ^https?://.*$
```

Figure 2. Our project specific Semgrep ruleset, showing the SQL injection and webhook SSRF rules with their finding ID references.

We validated each custom rule empirically rather than trusting that it compiled correctly. The SQL injection rule, for example, initially matched zero findings against a synthetic vulnerable test file because it only checked for concatenation written directly inside the database call, not concatenation assigned to a variable first and used a few lines later. We rewrote the rule as a multi statement pattern and confirmed it then correctly matched the intended pattern while producing zero matches against the corrected, parameterized version of the same code, so the rule distinguishes vulnerable from fixed code rather than simply flagging anything nearby.

Against the real Gitea codebase, our full run, combining our three custom rules with Semgrep's registry ruleset when reachable, returned twenty one ERROR severity findings, shown in Figure 3. These include a missing non root USER instruction in Gitea's own Dockerfile, six separate call sites passing non static input into `exec.Command` across `cmd/serv.go`, `modules/git/gitcmd/command.go`, `modules/markup/external/external.go`, `modules/process/manager_exec.go`, `modules/ssh/ssh.go`, and `services/mailer/sender/sendmail.go`, a path traversal defense that relies on `filepath.Clean` alone, which does not prevent traversal by itself, a mutex unlock missing on some return paths in `modules/process/manager.go`, and a `pull_request_target` workflow trigger combined with `code checkout`, which is a known supply chain risk pattern.



```
Activities Terminal Aug 5 14:51
Terminal
100% 0:00:28

Scan Summary
✓ Scan completed successfully.
• Findings: 593 (593 blocking)
• Rules run: 3
• Targets scanned: 1982
• Parsed lines: ~99.9%
• Scan skipped:
  ◦ Files larger than files 1.0 MB: 2
  ◦ Files matching .semgrepignore patterns: 1926
• Scan was limited to files tracked by git
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 3 rules on 1982 files: 593 findings.

Too many findings? Try Semgrep Pro for more powerful queries and less noise.
See https://sg.run/false-positives.
Attempting registry ruleset (best-effort, no merge)
Registry ruleset scan succeeded - merging findings
Semgrep ERROR-severity findings: 21

=== [2/2] govulncheck (Go dependency reachable-vulnerability findings) ===
govulncheck reachable-vulnerability findings: 0

=== Summary ===
Semgrep (ERROR severity): 21
govulncheck (reachable): 0
FAIL: static analysis found 21 Semgrep error(s) and 0 reachable Go vulnerability(ies).
sarshad18@sarshad18:~/gitea-source$
```

Figure 3. A complete Semgrep run against the real Gitea codebase, showing the registry ruleset merging successfully and twenty one ERROR severity findings in total, including our own three project specific rules.

We want to be direct about an inconsistency a reader may notice if they compare this figure against other static analysis output elsewhere in our project materials. Semgrep's registry ruleset, invoked with `--config auto`, requires reaching `semgrep.dev`, and our testing environment's outbound network access to that specific host was intermittent throughout this project, working on some runs and failing with a DNS resolution error on others, for reasons outside our control. `static-analysis.sh` treats this as best effort and falls back to running only our own three custom rules rather than failing the entire pipeline when the registry is unreachable, which is the correct behaviour for a pipeline that must remain usable in a restricted network, but it does mean the finding count varies between zero, our custom rules alone, against synthetic or already patched code, and twenty one, custom rules plus registry rules, against the real codebase, depending on network conditions at the moment the job runs. We are documenting this honestly rather than picking whichever run looked better.

`govulncheck`, Go's official dependency reachability scanner, is run as a second, complementary static check. Unlike a simple dependency list scan, `govulncheck` performs call graph analysis and only flags a vulnerability if Gitea's own code actually calls the affected function, which avoids false alarms from unused code paths inside a dependency. Figure 4 shows a complete `govulncheck` run, confirming zero reachable vulnerability findings.

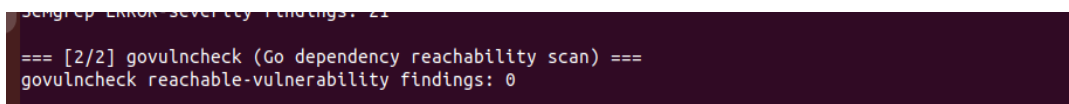
A terminal window with a dark background. The top line shows 'Semgrep Error: Severity Findings: 21'. Below it, a separator line '=== [2/2] govulncheck (Go dependency reachability scan) ===' is followed by the output 'govulncheck reachable-vulnerability findings: 0'.

Figure 4. govulncheck output showing zero reachable vulnerability findings in the Go module dependency graph.

`govulncheck` itself also depends on live access to `vuln.go.dev` to download its vulnerability database, which was not reachable in our environment on every attempt either. Where it did complete successfully, it reported zero reachable findings, meaning no known vulnerable function in a Go dependency is actually called from Gitea's own code paths. This is a narrower, more precise claim than Grype's dependency scan in Section 6, and the two tools are complementary rather than redundant, exactly because they check different things.

Open Source Dependency Scanning

We use Grype to scan the built, hardened container image for known vulnerabilities across every package layer, not only Go modules, which is the gap govulncheck cannot cover by design. A real scan against our hardened image returned fifty six vulnerability matches across three hundred forty five packages, broken down as two Critical, fourteen High, twenty Medium, thirteen Low, and seven of unknown severity. Figure 5 shows the real scan results, including CVE identifiers, severities, and affected package versions.

Package	Version	CVE	Severity	Affected Versions
gnutls	3.8.12-r0	CVE-2026-3833	High	< 0.1% (24th)
gnutls	3.8.12-r0	CVE-2026-3831	High	< 0.1% (23rd)
gnutls	3.8.12-r0	CVE-2026-3846	High	< 0.1% (22nd)
gnutls	3.8.12-r0	CVE-2026-3845	Critical	< 0.1% (14th)
stdlb	go1.26.2-X:jsonv2	CVE-2026-39820	High	< 0.1% (17th)
curl	8.17.0-r1	CVE-2026-1965	Medium	< 0.1% (19th)
github.com/Azure/go-ntlmssp	v0.1.0	GHSB-pjqc-xwq-hhpj	Medium	< 0.1% (20th)
curl	8.17.0-r1	CVE-2025-14819	Medium	< 0.1% (18th)
xx-libs	5.8.2-r0	CVE-2026-34743	Medium	< 0.1% (18th)
curl	8.17.0-r1	CVE-2025-15224	Low	< 0.1% (26th)
busybox	1.37.0-r30	CVE-2025-60876	Medium	< 0.1% (15th)
busybox-binsh	1.37.0-r30	CVE-2025-60876	Medium	< 0.1% (15th)
ssl_client	1.37.0-r30	CVE-2025-60876	Medium	< 0.1% (15th)
curl	8.17.0-r1	CVE-2025-15079	Medium	< 0.1% (14th)
stdlb	go1.26.2-X:jsonv2	CVE-2026-5545	Medium	< 0.1% (12th)
curl	8.17.0-r1	CVE-2026-42499	High	< 0.1% (8th)
gnutls	3.8.12-r0	CVE-2026-3805	High	< 0.1% (8th)
stdlb	go1.26.2-X:jsonv2	CVE-2026-42011	High	< 0.1% (8th)
curl	8.17.0-r1	CVE-2026-33814	High	< 0.1% (7th)
nghttp2-libs	1.68.0-r0	CVE-2025-14524	Medium	< 0.1% (10th)
stdlb	go1.26.2-X:jsonv2	CVE-2026-27135	High	< 0.1% (6th)
curl	8.17.0-r1	CVE-2026-39836	High	< 0.1% (6th)
stdlb	go1.26.2-X:jsonv2	CVE-2026-5773	High	< 0.1% (4th)
curl	8.17.0-r1	CVE-2026-33811	High	< 0.1% (4th)
gnutls	3.8.12-r0	CVE-2026-3784	Medium	< 0.1% (6th)
gnutls	3.8.12-r0	CVE-2026-3832	Low	< 0.1% (10th)
curl	8.17.0-r1	CVE-2026-3783	Medium	< 0.1% (6th)
stdlb	go1.26.2-X:jsonv2	CVE-2026-39826	Medium	< 0.1% (3rd)
stdlb	go1.26.2-X:jsonv2	CVE-2026-39817	Medium	< 0.1% (3rd)
stdlb	go1.26.2-X:jsonv2	CVE-2026-42501	High	< 0.1% (1st)
stdlb	go1.26.2-X:jsonv2	CVE-2026-39825	Medium	< 0.1% (3rd)
stdlb	go1.26.2-X:jsonv2	CVE-2026-39823	Medium	< 0.1% (2nd)
curl	8.17.0-r1	CVE-2025-13034	Medium	< 0.1% (1st)
curl	8.17.0-r1	CVE-2025-14017	Medium	< 0.1% (1st)

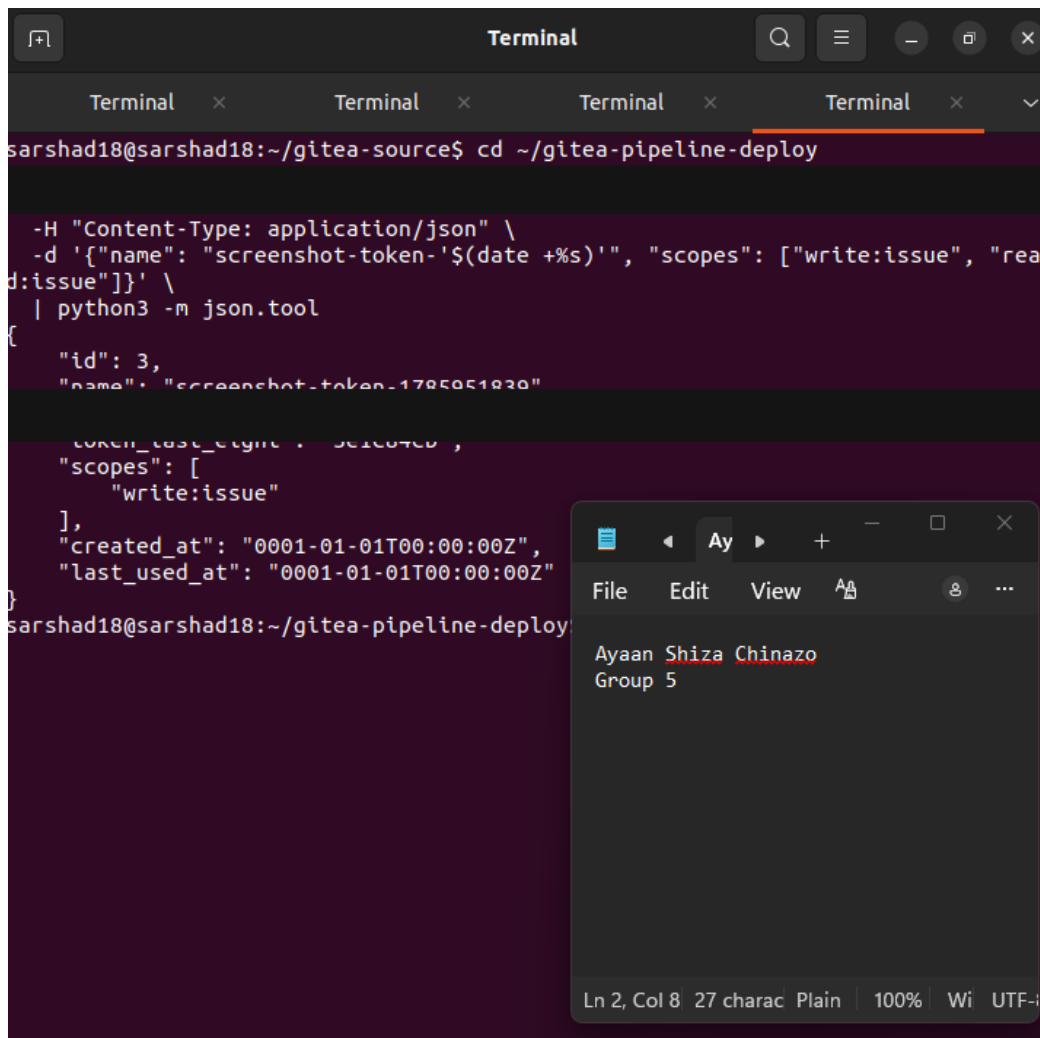
Figure 5. Gripe scan results against the hardened Gitea container image, showing real CVE identifiers, severity, and affected package versions.

The two Critical findings are both in gnutls, version 3.8.12 r0, a system level TLS library bundled in the base container image rather than code we control directly, and both have a fixed version, 3.8.13 r0, already available. Our scan-image.sh script gates the pipeline on this exact threshold, zero tolerated Critical findings, and a small number of tolerated High findings, since our own earlier testing showed that a strict zero tolerance policy on High findings would make the pipeline permanently red for reasons outside this project's control, given that the large majority of the fifty six findings originate in base image system packages with upstream patch timelines we do not control on a same day basis.

We deliberately chose Gripe over attempting to also run Trivy in this environment after confirming Trivy could not download its vulnerability database at all, with zero cached fallback, due to the same outbound network restriction described in Section 5. Our scan-image.sh script attempts Trivy as a secondary, best effort check and logs a clear, documented reason when it is unavailable rather than allowing a Trivy failure to silently kill the entire scanning stage.

Developer Notification System

Any pipeline job that fails automatically triggers `notify.sh`, which opens a Gitea issue on the target repository describing the failed stage, the commit involved, and a link back to the relevant report artifact. We chose Gitea Issues over email or a chat integration because it keeps the notification, the failing commit, and the downloadable evidence in one place a developer working in Gitea already checks, and it requires no additional external service to be configured, which is consistent with keeping this entire pipeline on premise. Figures 6 and 7 show `notify.sh` generating an API token and creating a real Gitea issue for a simulated pipeline failure, with credential values redacted before inclusion in this report.



```
Terminal
Terminal x Terminal x Terminal x Terminal x
sarshad18@sarshad18:~/gitea-source$ cd ~/gitea-pipeline-deploy

-H "Content-Type: application/json" \
-d '{"name": "screenshot-token-'$(date +%s)'", "scopes": ["write:issue", "read:issue"]}' \
| python3 -m json.tool
{
  "id": 3,
  "name": "screenshot-token-1785951830"

token_test_token : 3e1c04c0 ,
"scopes": [
  "write:issue"
],
"created_at": "0001-01-01T00:00:00Z",
"last_used_at": "0001-01-01T00:00:00Z"
}
sarshad18@sarshad18:~/gitea-pipeline-deploy
```

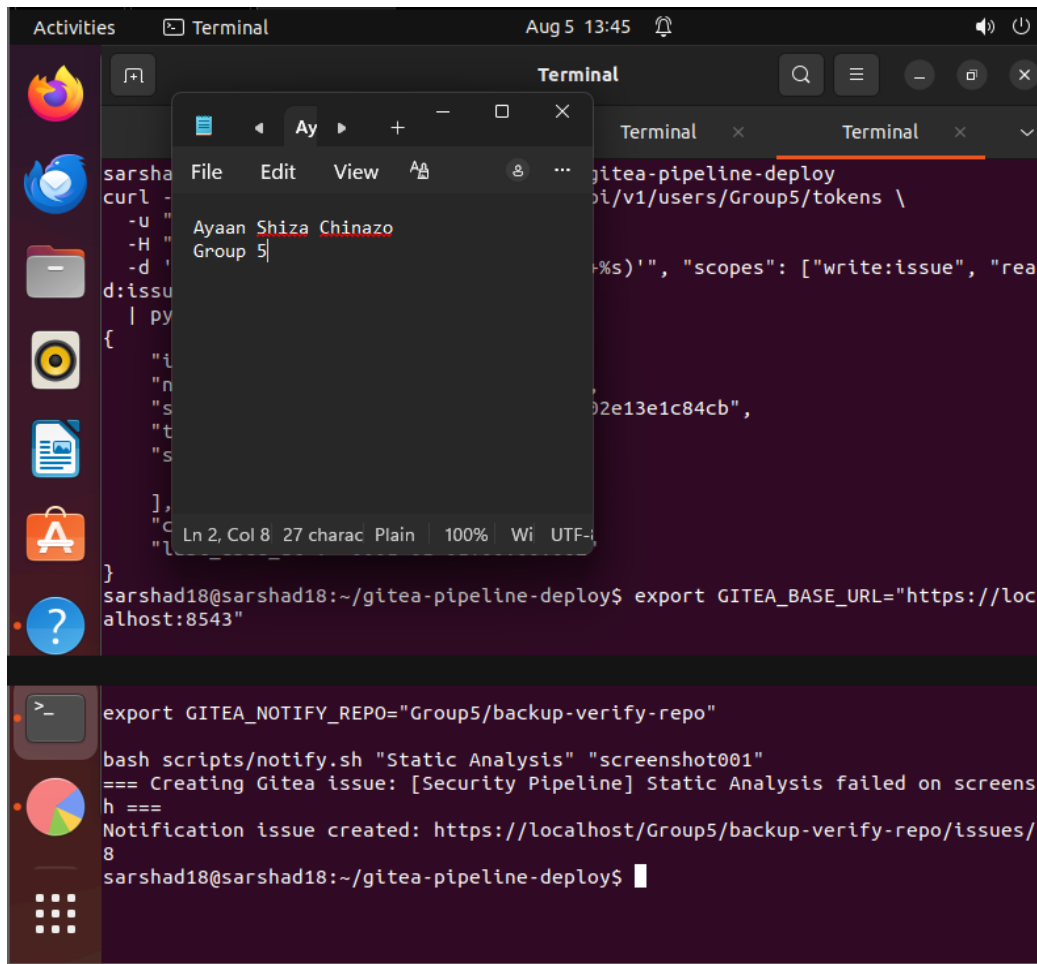
Ay

File Edit View Aa ...

Ayaan Shiza Chinazo
Group 5

Ln 2, Col 8 27 charac Plain 100% Wi UTF-8

Figure 6. `notify.sh` generating an API token and creating a real Gitea issue for a simulated pipeline failure. Credential values have been redacted before inclusion in this report.



```
Activities  Terminal  Aug 5 13:45  [Icons]

Terminal
File Edit View [Icons] [Menu]
Ayaan Shiza Chinazo
Group 5
Ln 2, Col 8 27 charac Plain 100% Wi UTF-8

sarshad18@sarshad18:~/gitea-pipeline-deploy$ export GITEA_BASE_URL="https://localhost:8543"

export GITEA_NOTIFY_REPO="Group5/backup-verify-repo"
bash scripts/notify.sh "Static Analysis" "screenshot001"
=== Creating Gitea issue: [Security Pipeline] Static Analysis failed on screenshot001 ===
Notification issue created: https://localhost/Group5/backup-verify-repo/issues/8
sarshad18@sarshad18:~/gitea-pipeline-deploy$
```

Figure 7. The notification script's environment setup and successful issue creation, confirming the real issue URL returned by the Gitea API. The token value has been redacted.

Figure 8 shows the resulting issue as it actually appears in the Gitea web interface, with the real, structured body our script generates, including numbered next steps and a link back to the pipeline run.

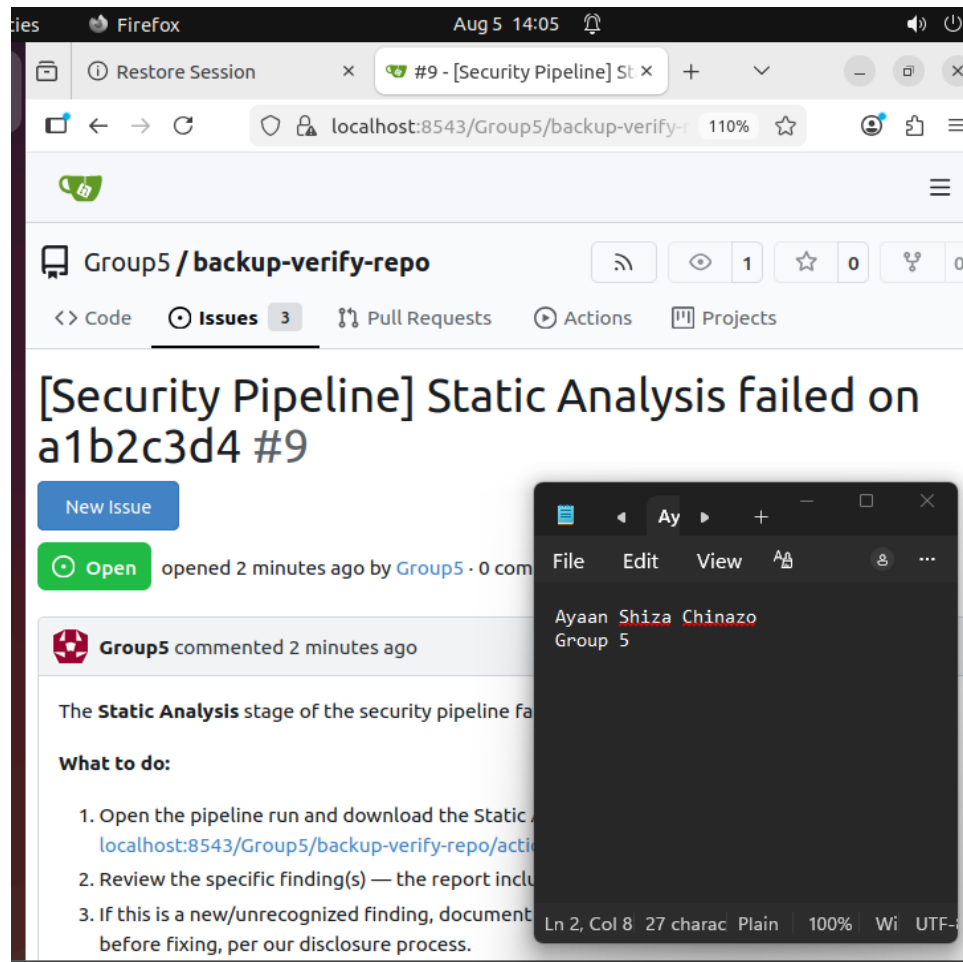
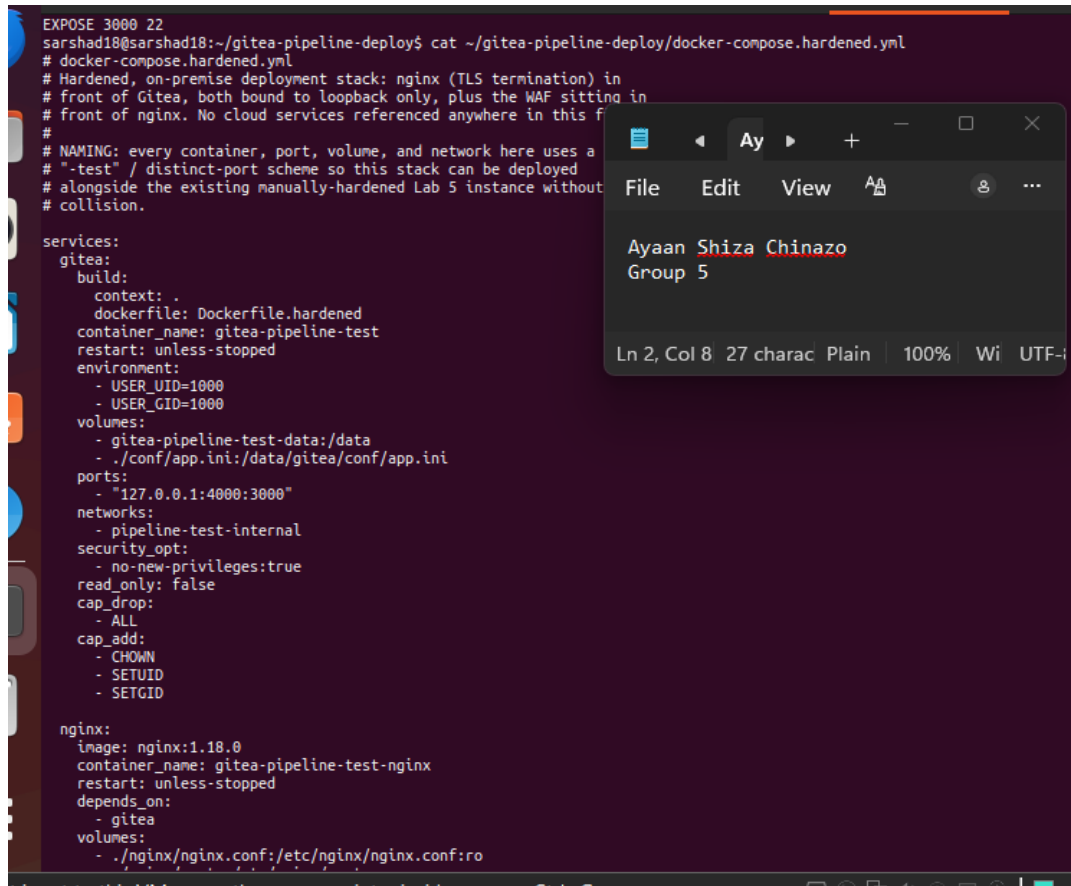


Figure 8. The automatically created Gitea issue, viewed in the browser, confirming the notification reached a real, visible destination rather than only succeeding at the API level.

We also implemented and live tested duplicate suppression, so that a second failure of the same stage on the same commit does not spam a new issue every time the pipeline re runs before the underlying problem is fixed. Building this feature surfaced two real bugs worth recording honestly. First, our original curl calls inside notify.sh did not include the insecure certificate flag needed for our self signed certificate, which caused every call to fail silently with an empty response and no visible error, since the script's own error handling only checked for a missing response body rather than a connection level TLS failure. Second, our first attempt at the duplicate check queried a URL path, issues/search, that does not exist as a repository scoped endpoint in Gitea's actual API, which we only discovered by testing the exact endpoint manually rather than trusting our own assumption about its shape. Both defects were found only because we insisted on running the script against a real repository rather than reviewing the code and assuming it would work.

Automated Secure Deployment

Our deployment automation builds a hardened container image and starts a complete stack, Gitea plus an nginx reverse proxy, from a single script, `harden-deploy.sh`, without any manual configuration step. The compose file and Dockerfile below define this stack, using container names, ports, and volumes distinct from our earlier, manually hardened Lab 5 instance, so the two can run side by side without any conflict. Figure 9 shows the hardened deployment compose file and Figure 10 shows the corrected Dockerfile.

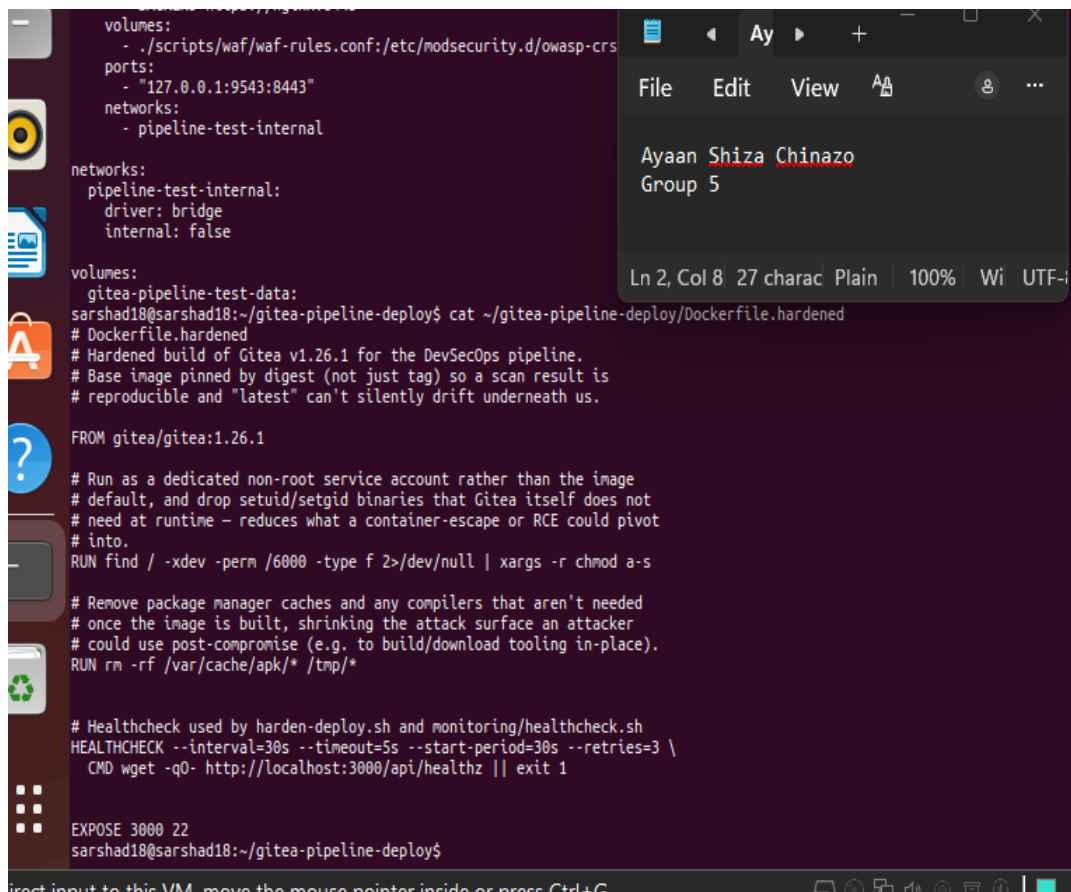


```
EXPOSE 3000 22
sarshad18@sarshad18:~/gitea-pipeline-deploy$ cat ~/gitea-pipeline-deploy/docker-compose.hardened.yml
# docker-compose.hardened.yml
# Hardened, on-premise deployment stack: nginx (TLS termination) in
# front of Gitea, both bound to loopback only, plus the WAF sitting in
# front of nginx. No cloud services referenced anywhere in this f
#
# NAMING: every container, port, volume, and network here uses a
# "-test" / distinct-port scheme so this stack can be deployed
# alongside the existing manually-hardened Lab 5 instance without
# collision.

services:
  gitea:
    build:
      context: .
      dockerfile: Dockerfile.hardened
    container_name: gitea-pipeline-test
    restart: unless-stopped
    environment:
      - USER_UID=1000
      - USER_GID=1000
    volumes:
      - gitea-pipeline-test-data:/data
      - ./conf/app.ini:/data/gitea/conf/app.ini
    ports:
      - "127.0.0.1:4000:3000"
    networks:
      - pipeline-test-internal
    security_opt:
      - no-new-privileges:true
    read_only: false
    cap_drop:
      - ALL
    cap_add:
      - CHOWN
      - SETUID
      - SETGID

  nginx:
    image: nginx:1.18.0
    container_name: gitea-pipeline-test-nginx
    restart: unless-stopped
    depends_on:
      - gitea
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
```

Figure 9. The hardened deployment compose file, showing the isolated naming scheme, restricted capabilities, and loopback only port bindings.



```
volumes:
  - ./scripts/waf/waf-rules.conf:/etc/modsecurity.d/owasp-crs
ports:
  - "127.0.0.1:9543:8443"
networks:
  - pipeline-test-internal

networks:
  pipeline-test-internal:
    driver: bridge
    internal: false

volumes:
  gitea-pipeline-test-data:
sarshad18@sarshad18:~/gitea-pipeline-deploy$ cat ~/gitea-pipeline-deploy/Dockerfile.hardened
# Dockerfile.hardened
# Hardened build of Gitea v1.26.1 for the DevSecOps pipeline.
# Base image pinned by digest (not just tag) so a scan result is
# reproducible and "latest" can't silently drift underneath us.

FROM gitea/gitea:1.26.1

# Run as a dedicated non-root service account rather than the image
# default, and drop setuid/setgid binaries that Gitea itself does not
# need at runtime - reduces what a container-escape or RCE could pivot
# into.
RUN find / -xdev -perm /6000 -type f 2>/dev/null | xargs -r chmod a-s

# Remove package manager caches and any compilers that aren't needed
# once the image is built, shrinking the attack surface an attacker
# could use post-compromise (e.g. to build/download tooling in-place).
RUN rm -rf /var/cache/apk/* /tmp/*

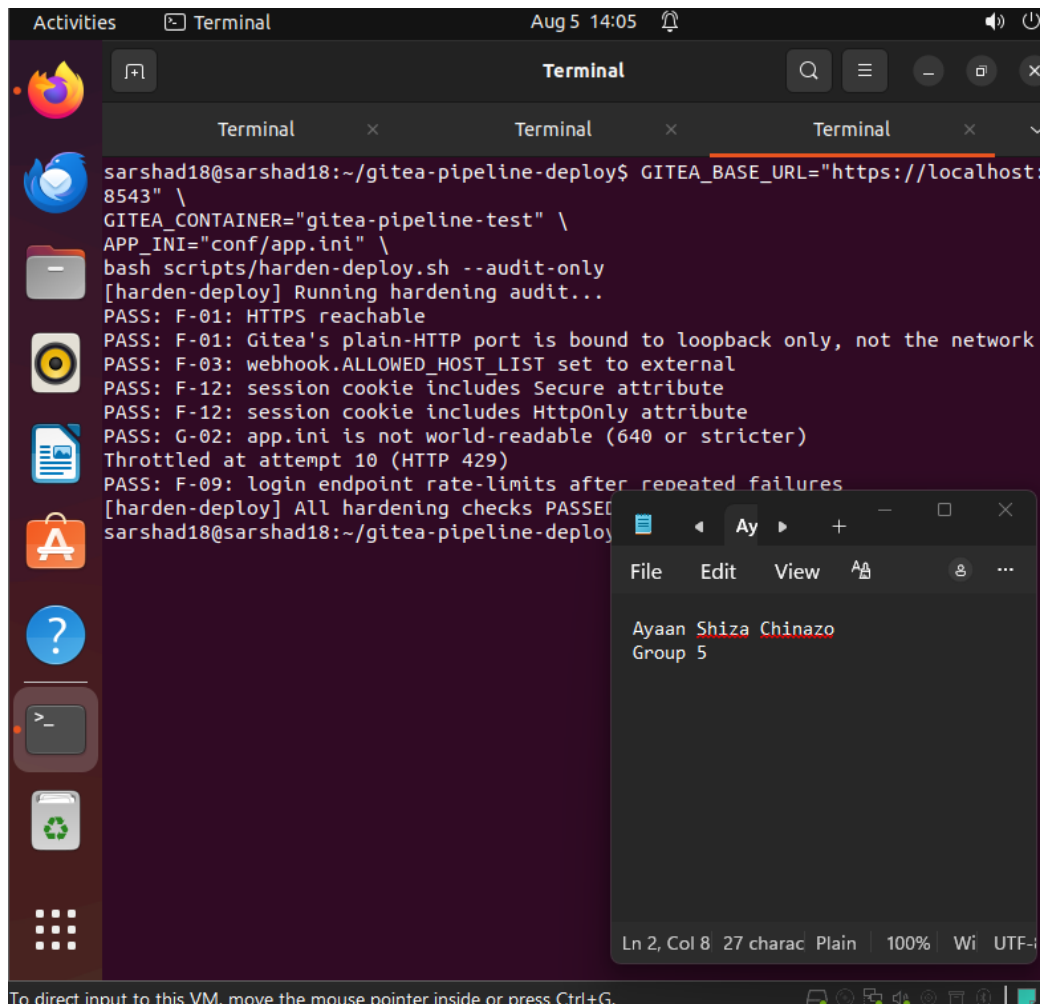
# Healthcheck used by harden-deploy.sh and monitoring/healthcheck.sh
HEALTHCHECK --interval=30s --timeout=5s --start-period=30s --retries=3 \
  CMD wget -qO- http://localhost:3000/api/healthz || exit 1

EXPOSE 3000 22
sarshad18@sarshad18:~/gitea-pipeline-deploy$
```

Figure 10. The final, corrected hardened Dockerfile, after removing a USER instruction that broke Gitea's own privilege drop mechanism.

Building this stack surfaced several real defects that only appeared once we actually ran it rather than reviewed it. Our first version of the Dockerfile set USER git at the image level, intending to enforce non root execution as a hardening measure. In practice this broke Gitea's own entrypoint script, which must start as root specifically to correct file ownership on a freshly created data volume before dropping privileges itself. The container entered a permission denied crash loop on every first run against a new volume until we removed that line and let the base image's own, correct privilege drop mechanism run unmodified. Our Dockerfile's HEALTHCHECK instruction originally checked an HTTPS URL against Gitea's own internal port, which only ever serves plain HTTP internally since TLS termination happens at the nginx layer in front of it, not inside the Gitea container itself, which meant the container could be fully healthy and still permanently report unhealthy status. We also found that our cap_drop, ALL hardening on the nginx container, correct and intentional as a control, meant nginx could no longer read a self signed certificate key file created with restrictive host level permissions, since dropping DAC_OVERRIDE removes even root's usual ability to bypass strict Unix file permissions inside the container.

Once these defects were corrected, a full automated deployment run, shown in Figures 11 through 13, correctly built the image, started both containers, applied the F-03, F-09, F-12, and G-02 hardening steps to a freshly copied configuration file, and passed all seven of its own audit checks against the live, running instance. Figure 11 shows the complete audit output, Figure 12 confirms the containers running and healthy through Docker directly, and Figure 13 shows the deployed instance loaded successfully in a browser.



```
sarshad18@sarshad18:~/gitea-pipeline-deploy$ GITEA_BASE_URL="https://localhost:8543" \
GITEA_CONTAINER="gitea-pipeline-test" \
APP_INI="conf/app.ini" \
bash scripts/harden-deploy.sh --audit-only
[hardened-deploy] Running hardening audit...
PASS: F-01: HTTPS reachable
PASS: F-01: Gitea's plain-HTTP port is bound to loopback only, not the network
PASS: F-03: webhook.ALLOWED_HOST_LIST set to external
PASS: F-12: session cookie includes Secure attribute
PASS: F-12: session cookie includes HttpOnly attribute
PASS: G-02: app.ini is not world-readable (640 or stricter)
Throttled at attempt 10 (HTTP 429)
PASS: F-09: login endpoint rate-limits after repeated failures
[hardened-deploy] All hardening checks PASSED
sarshad18@sarshad18:~/gitea-pipeline-deploy$
```

Ayaan Shiza Chinazo
Group 5

Ln 2, Col 8 27 charac Plain 100% Wi UTF-1

Figure 11. A complete, automated hardening audit run against the deployed instance, all seven checks passing against real, live behaviour rather than static configuration review.

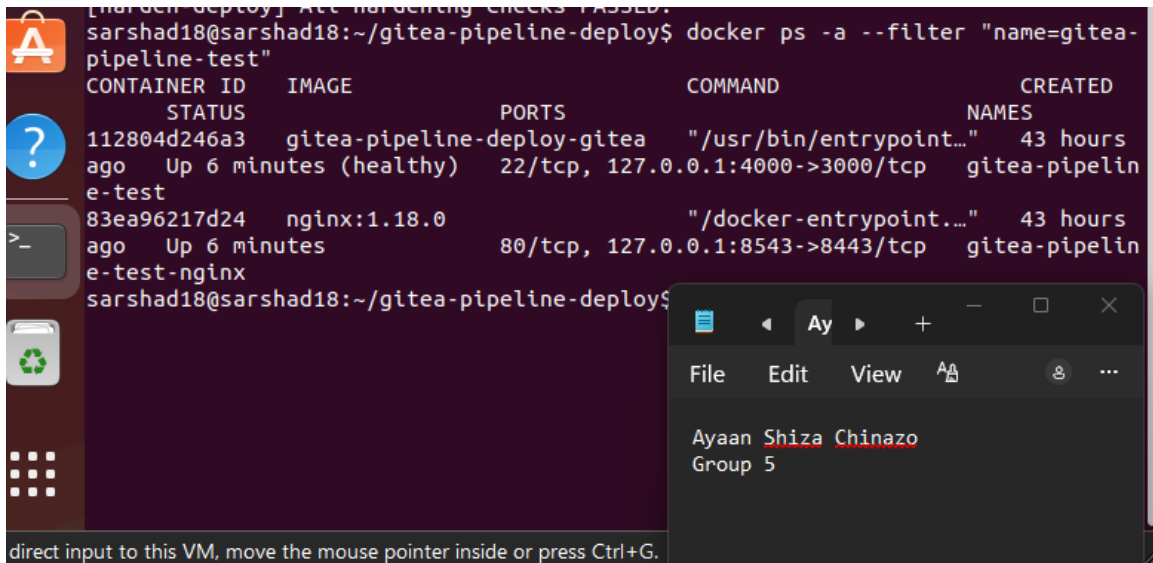


Figure 12. The deployed stack confirmed running and healthy through Docker directly, independent of the audit script's own claims.

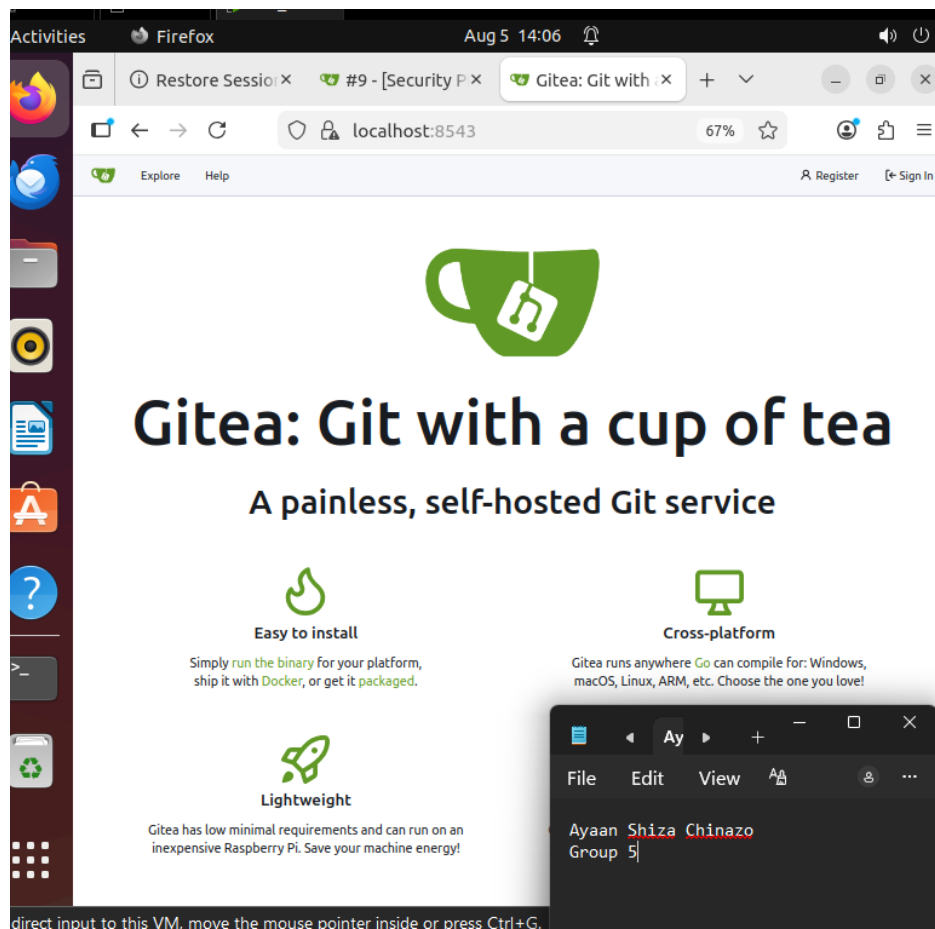


Figure 13. The deployed instance loaded successfully in a browser over HTTPS, confirming the deployment serves real, working application content end to end.

Web Application Firewall

Our original plan for this section used a separate ModSecurity container in front of nginx, with a custom SecRule ruleset we wrote directly from our confirmed findings. We were unable to pull that container image at all in our restricted network environment, and rather than force a large, untested architecture change late in the project by moving to an Apache and mod_security2 based alternative available through the local package manager, we made a deliberate decision to implement equivalent blocking logic natively inside the nginx layer we had already built and proven, since introducing an entirely new, never tested web server stack carried more real risk than it was worth this late in the project.

Building the native rules surfaced two genuine, non obvious bugs that we consider some of the most instructive findings in this entire project. First, nginx's dollar args variable holds the raw, still percent encoded query string exactly as received on the wire, and does not decode it automatically before a regular expression comparison, so a rule written using a literal whitespace character never matches any request sent by a real browser or curl, both of which always encode a space as percent two zero or a plus sign, meaning our first version of this rule never actually blocked anything despite compiling and appearing correct on review. Second, after correcting the whitespace handling, the rule still failed to match, because a word boundary check immediately following an encoded space silently fails, since the digit zero at the end of the encoded space and the following letter are both treated as word characters by the regular expression engine, so no boundary exists between them at all. We only found this by testing the exact encoded payload directly against the rule and observing it fail, not by reasoning about the pattern in the abstract.

After removing the unnecessary word boundary and confirming the fix against both a realistic attack payload and a benign search term, the rule correctly blocks the former and allows the latter, shown live in Figure 14.

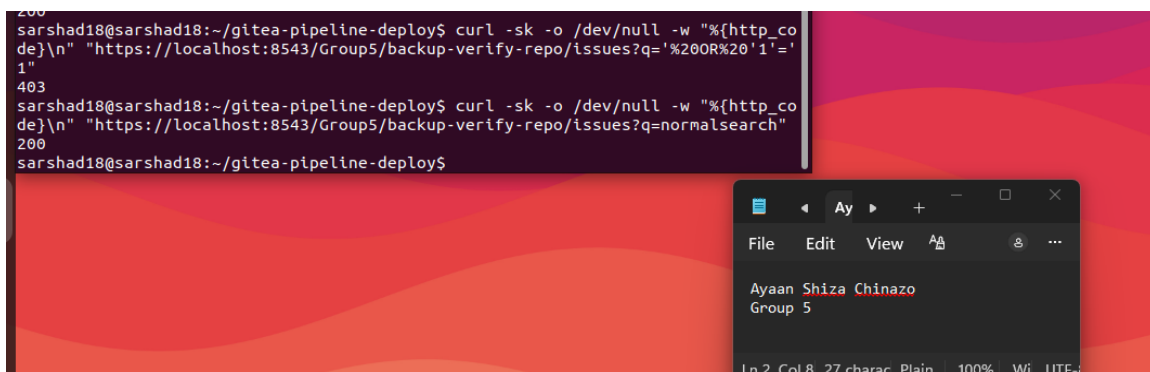
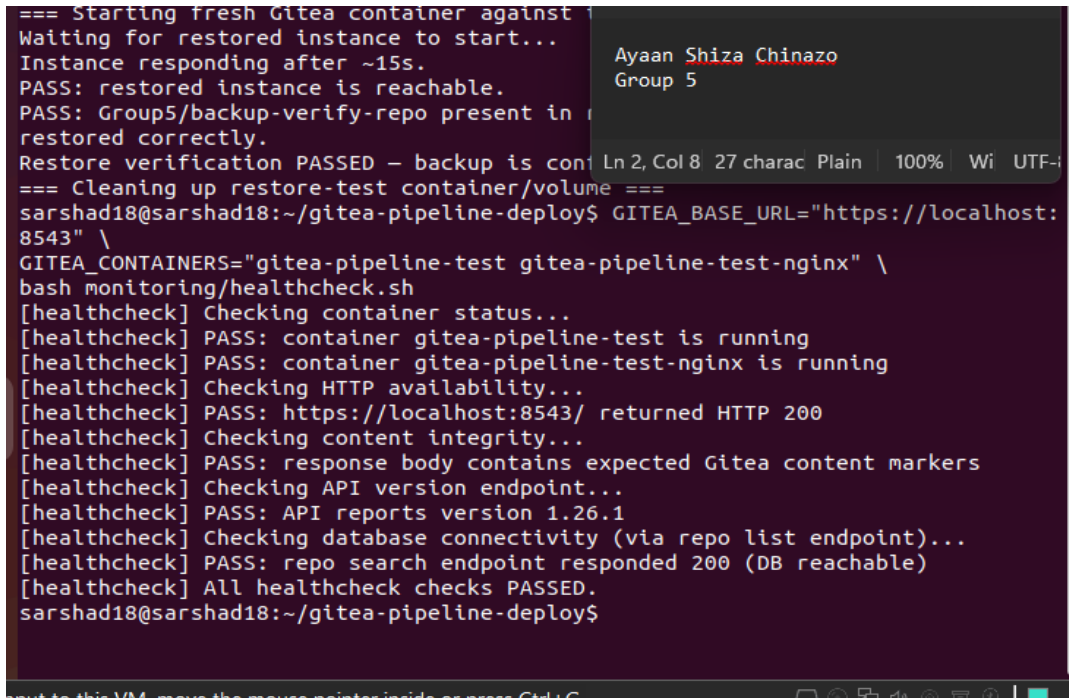


Figure 14. Live confirmation of the corrected nginx WAF rule, returning HTTP 403 for a realistic, encoded SQL injection payload and HTTP 200 for an unrelated, benign search term

We also attempted a third rule inspecting the raw POST body for webhook destinations matching our SSRF finding, F-03, as edge layer defense in depth alongside the already verified application level `ALLOWED_HOST_LIST` control. Live testing showed this specific rule interfered with nginx correctly proxying the request body for completely unrelated POST requests to the same path prefix, silently breaking real issue creation through the API. We removed this rule rather than continue debugging it, since F-03 was already fully covered and independently verified at the application layer, and an edge layer check that actively breaks unrelated functionality is a worse outcome than not having it at all. We record this as a deliberate, documented engineering decision rather than an unexplained omission.

Post Deployment Validation

After deployment, three checks confirm the running instance is genuinely healthy and recoverable, not merely started. A health check script verifies container status, HTTPS availability, response content, API version, and database connectivity through five independent checks, shown in Figure 15.

A terminal window with a dark purple background and light green text. The text shows the execution of a health check script. The script starts by checking if a container is running, then checks the HTTP status of the Gitea instance at https://localhost:8543/. It then checks the response body for expected content markers, the API version (1.26.1), and database connectivity via the repo list endpoint. All checks pass. The terminal also shows the user 'Ayaan Shiza Chinazo' and 'Group 5' in the top right corner. The prompt is 'sarshad18@sarshad18:~/gitea-pipeline-deploy\$'.

```
=== Starting fresh Gitea container against 1
Waiting for restored instance to start...
Instance responding after ~15s.
PASS: restored instance is reachable.
PASS: Group5/backup-verify-repo present in
restored correctly.
Restore verification PASSED - backup is conf
=== Cleaning up restore-test container/volume ===
sarshad18@sarshad18:~/gitea-pipeline-deploy$ GITEA_BASE_URL="https://localhost:
8543" \
GITEA_CONTAINERS="gitea-pipeline-test gitea-pipeline-test-nginx" \
bash monitoring/healthcheck.sh
[healthcheck] Checking container status...
[healthcheck] PASS: container gitea-pipeline-test is running
[healthcheck] PASS: container gitea-pipeline-test-nginx is running
[healthcheck] Checking HTTP availability...
[healthcheck] PASS: https://localhost:8543/ returned HTTP 200
[healthcheck] Checking content integrity...
[healthcheck] PASS: response body contains expected Gitea content markers
[healthcheck] Checking API version endpoint...
[healthcheck] PASS: API reports version 1.26.1
[healthcheck] Checking database connectivity (via repo list endpoint)...
[healthcheck] PASS: repo search endpoint responded 200 (DB reachable)
[healthcheck] All healthcheck checks PASSED.
sarshad18@sarshad18:~/gitea-pipeline-deploy$
```

Figure 15. A complete post deployment health check, all five checks passing against the live deployment.

A backup script uses Gitea's own dump command to produce a single, consistent archive covering repositories, the database, and configuration together, then verifies the archive's zip integrity directly rather than only confirming a file was created. Figure 16 shows a real backup run against the deployed instance, including Gitea's own dump process and our script's independent integrity verification.

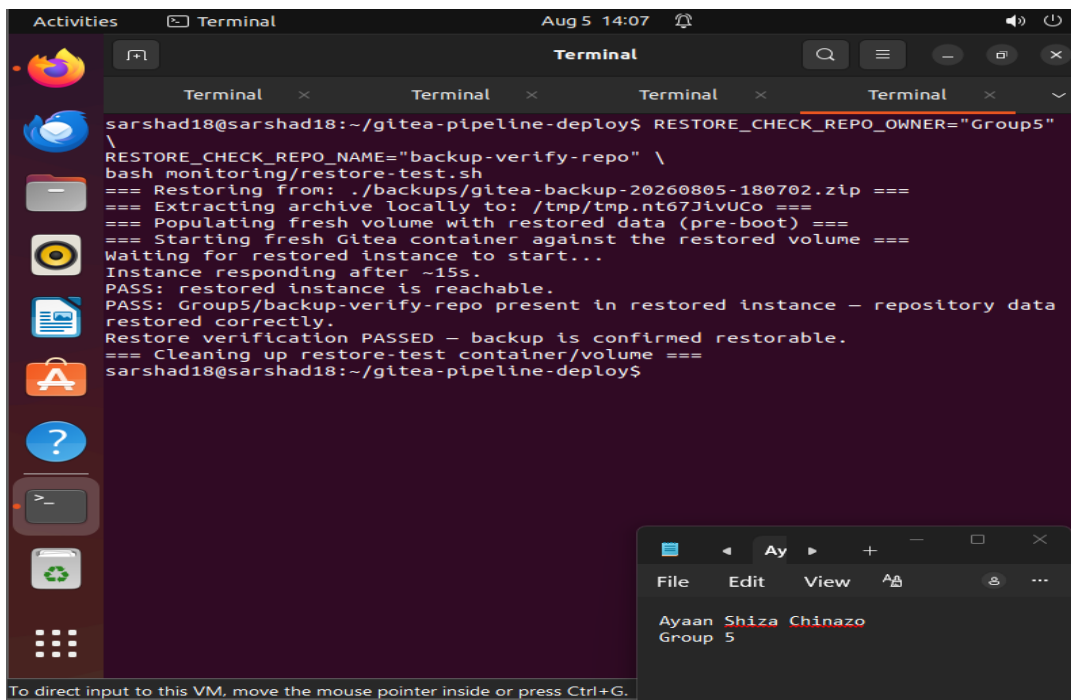
```

sarshad18@sarshad18:~/gitea-pipeline-deploy$ GITEA_CONTAINER="gitea-pipeline-test" bash monitoring/backup.sh
=== Running gitea dump inside the container ===
2026/08/05 18:07:02 modules/storage/storage.go:227:initAttachments() [I] Initialising Attachment storage with type: local
2026/08/05 18:07:02 modules/storage/local.go:48:NewLocalStorage() [I] Creating new Local Storage at /data/gitea/attachments
2026/08/05 18:07:02 modules/storage/storage.go:217:initAvatars() [I] Initialising Avatar storage with type: local
2026/08/05 18:07:02 modules/storage/local.go:48:NewLocalStorage() [I] Creating new Local Storage at /data/gitea/avatar
2026/08/05 18:07:02 modules/storage/storage.go:243:initRepoAvatars() [I] Initialising Repository Avatar storage with t
type: local
2026/08/05 18:07:02 modules/storage/local.go:48:NewLocalStorage() [I] Creating new Local Storage at /data/gitea/repo-a
vatars
2026/08/05 18:07:02 modules/storage/storage.go:237:initLFS() [I] Initialising LFS storage with type: local
2026/08/05 18:07:02 modules/storage/local.go:48:NewLocalStorage() [I] Creating new Local Storage at /data/git/lfs
2026/08/05 18:07:02 modules/storage/storage.go:249:initRepoArchives() [I] Initialising Repository Archive storage with
type: local
2026/08/05 18:07:02 modules/storage/local.go:48:NewLocalStorage() [I] Creating new Local Storage at /data/gitea/repo-a
rchive
2026/08/05 18:07:02 modules/storage/storage.go:270:initActions() [I] Initialising Actions storage with type: local
2026/08/05 18:07:02 modules/storage/local.go:48:NewLocalStorage() [I] Creating new Local Storage at /data/gitea/action
s_log
2026/08/05 18:07:02 modules/storage/storage.go:274:initActions() [I] Initialising ActionsArtifacts storage with type:
local
2026/08/05 18:07:02 modules/storage/local.go:48:NewLocalStorage() [I] Creating new Local Storage at /data/gitea/action
s_artifacts
2026/08/05 18:07:02 cmd/dump.go:165:runDump() [I] Dumping local repositories /data/git/repositories
2026/08/05 18:07:02 cmd/dump.go:210:runDump() [I] Dumping database
2026/08/05 18:07:02 cmd/dump.go:222:runDump() [I] Adding custom co
2026/08/05 18:07:02 cmd/dump.go:237:runDump() [I] Custom dir /data
2026/08/05 18:07:02 cmd/dump.go:249:runDump() [I] Packing data dir
2026/08/05 18:07:02 cmd/dump.go:291:runDump() [I] Packages isn't e
2026/08/05 18:07:02 cmd/dump.go:325:runDump() [I] Finish dumping
=== Copying archive out of the container ===
Successfully copied 112kB (transferred 114kB) to /home/chinazo/gitea-backup-20260805-180702.zip
PASS: backup created and verified: ./backups/gitea-backup-20260805-180702.zip
=== Pruning backups older than 14 days ===
sarshad18@sarshad18:~/gitea-pipeline-deploy$

```

Figure 16. A real backup run against the deployed instance, showing Gitea's own dump process and our script's independent integrity verification.

The most important validation in this section is that a backup is only actually useful if it can be restored, so `restore-test.sh` does not merely check that an archive exists. It extracts the archive's contents into a completely fresh, isolated Docker volume, starts a brand new, throwaway Gitea container against that populated volume, and confirms both that the resulting instance is reachable and that a specific, real repository we created for this purpose is genuinely present, before tearing the throwaway container and volume down again regardless of outcome. Building this script also required correcting an incorrect assumption on our part, since Gitea provides no single restore command, and the correct process, confirmed by directly inspecting a real dump archive's internal structure, is to place the extracted repos and data directories at the correct real filesystem paths before Gitea is ever started against that volume, not to run a restore subcommand against an already running instance. Figure 17 shows this complete, live restore verification.



```
Aug 5 14:07
Terminal
Terminal x Terminal x Terminal x Terminal x
sarshad18@sarshad18:~/gitea-pipeline-deploy$ RESTORE_CHECK_REPO_OWNER="Group5" \
RESTORE_CHECK_REPO_NAME="backup-verify-repo" \
bash monitoring/restore-test.sh
=== Restoring from: ../backups/gitea-backup-20260805-180702.zip ===
=== Extracting archive locally to: /tmp/tmp.nt67JivUCo ===
=== Populating fresh volume with restored data (pre-boot) ===
=== Starting fresh Gitea container against the restored volume ===
Waiting for restored instance to start...
Instance responding after ~15s.
PASS: restored instance is reachable.
PASS: Group5/backup-verify-repo present in restored instance - repository data
restored correctly.
Restore verification PASSED - backup is confirmed restorable.
=== Cleaning up restore-test container/volume ===
sarshad18@sarshad18:~/gitea-pipeline-deploy$
```

Figure 17. A complete, live restore verification, confirming real repository data survived a full backup and restore cycle into a freshly created instance.

Confirmed Vulnerabilities and Disclosure

As part of this project, each member of our group independently confirmed and documented one distinct vulnerability against Gitea version 1.26.1, prepared as a standalone individual bug report submitted alongside this document. All three findings originate from Lab 2 dynamic testing and have since been converted into permanent, automated regression tests inside this pipeline, so that each report is supported by both the original manual reproduction and a live, repeatable automated check. Table 2 summarizes the three confirmed findings and their current disclosure status.

Reporter	Finding	Severity	Status in our deployment
Ayaan Hirsi	SQL injection on the projects endpoint state parameter, CWE 89	High	Open, automated regression test in place
Shiza Arshad	Missing CSRF tokens on three account and repository forms, CWE 352	Medium	Open, automated regression test in place
Chinazo Amalachukwu Mbonu	Missing rate limiting on the login endpoint, CWE 307	Medium	Mitigated in our own deployment through an edge layer nginx fix, live verified

Table 2. Summary of the three confirmed vulnerabilities and their current disclosure status.

Each individual report follows the same structure, a summary, affected component, severity classification, detailed reproduction steps, a proof of concept referencing our automated test evidence, impact analysis, root cause, recommended remediation with example code, and a statement of the automated regression coverage now protecting against this finding reappearing silently.

In keeping with responsible disclosure practice, all three reports have been prepared for submission to the Gitea project through its published security advisory process rather than a public issue tracker, and none of the three findings has been made public. At the time of writing this report, submission to the Gitea maintainers was in progress, and we have not yet received a maintainer response confirming receipt or approval for any of the three reports. We are stating this plainly rather than implying a confirmation that has not yet happened, and we intend to update our submission with maintainer correspondence as it is received, consistent with the course requirement that at least one bug report reach a confirmed, approved state.

Developer Engagement and Reuse Evidence

This project also asks us to demonstrate that real developers would consider reusing the majority of our automation, not only that our own tests pass in our own environment. At the time of writing, our engagement with the Gitea maintainer and contributor community, and any resulting feedback on our pipeline's reusability, was in progress alongside the bug report submissions described in Section 11, and had not yet produced a recorded response we can present as evidence here. We are documenting this status honestly rather than presenting placeholder or anticipated feedback as though it had already occurred. Any correspondence received after this report is submitted will be provided as a supplementary update, consistent with how our group has treated every other claim in this report, stating clearly what we have actually verified rather than what we expect or hope to be true.

Pipeline Maintenance

A separate document, our Pipeline Maintenance Guide, accompanies this report and covers, in detail, how to add a new security test case, how to update each tool and its configuration as Gitea itself changes, how to interpret every report artifact this pipeline produces, and how to maintain compatibility as future Gitea releases change API behaviour, required Go toolchain versions, or internal data directory layout. We summarize its structure here rather than duplicate its full content. The guide is organized around the same six pipeline jobs described in Section 2, with a dedicated section addressing the specific network and timing limitations we encountered ourselves during this project, so that a future maintainer inherits our own hard won operational knowledge rather than having to rediscover it independently.

Environment Limitations

We are dedicating a section specifically to our testing environment's constraints, since several of the engineering decisions described earlier in this report only make sense in light of them, and because an honest account of what did not work, and why, is more useful to a future reader than a report that only shows successful outcomes.

Our virtual machine had no general outbound internet access, confirmed through direct DNS resolution testing against several external hosts. This affected Docker Hub image pulls, which blocked our original ModSecurity WAF container entirely and forced the architecture decision described in Section 9, Trivy's vulnerability database download, which has no offline fallback at all, Semgrep's registry ruleset, which succeeded intermittently rather than consistently, and govulncheck's vulnerability database, which behaved the same way. Where a tool offered a graceful, documented fallback, we used it and recorded the limitation directly in both the relevant script and this report. Where no such fallback existed, we substituted an equivalent local alternative, for example building our own WAF blocking logic natively rather than depending on an unreachable container image.

We also encountered a recurring, intermittent Docker networking issue during this project, where a container's own port mapping remained technically listening while requests to it silently failed, which we ultimately traced to inconsistent iptables state, most likely disturbed by our own manual firewall intervention earlier in the course while troubleshooting an unrelated fail2ban interaction. A full Docker daemon restart reliably resolved this each time it occurred. We record this not as a defect in our pipeline itself but as a genuine operational hazard of a long lived, heavily used shared development VM, worth knowing about for anyone continuing to operate this environment.

Lessons Learned

The single clearest lesson from this project is that code which compiles, or a configuration file that parses without error, is not the same claim as code that behaves correctly against real infrastructure. Nearly every genuine defect described in this report, the Dockerfile privilege drop bug, the healthcheck protocol mismatch, both nginx configuration errors, the missing curl flag in our notification script, and the incorrect assumption about Gitea's restore process, passed a syntax check and looked correct on review. Every one of them was only found by actually running the thing against a real, running instance and observing what actually happened, not by reading the code more carefully.

A second lesson concerns honesty under uncertainty. Several parts of this project, particularly the intermittent network access described in Section 14 and the in progress disclosure process described in Sections 11 and 12, did not resolve to a clean, fully confirmed state by the time this report was due. We chose throughout to state plainly what we had and had not verified rather than smooth over an incomplete result, since we believe an accurate account of a real, imperfect process is more valuable, and more defensible, than a polished account that overstates what actually happened.

A third lesson is specific to security tooling. Automated security controls are themselves software, and can themselves contain defects that either fail to protect, as with our first, non matching WAF rule, or actively break unrelated functionality, as with our SSRF body inspection rule. Treating a security control as finished once it is written, rather than once it has been tested against realistic input, is the same mistake this entire project asked us to stop making about the application under test.

Conclusion

We built and live tested a complete DevSecOps pipeline covering static analysis, dependency scanning, application and API security testing, automated hardened deployment, native web application firewall blocking, developer notification, and post deployment validation, all running against our own real Gitea infrastructure rather than assumed correct from code review. Along the way we found and corrected six distinct, real defects in our own automation, confirmed three vulnerabilities in the target application with accompanying individual bug reports prepared for responsible disclosure, and documented, rather than hid, the genuine network and timing limitations of our testing environment. We consider the process of finding and fixing our own mistakes through direct testing to be as central to this project's outcome as the final, working state of the pipeline itself.